

Classes, Objects, and Interfaces

Overview and objectives

- **Course:** CIS 3300, Intermediate Object-Oriented Programming (Java)
- **Texts:** *Core Java, Volume I* (13th ed.), Chapters 4 and 6.1
- **Prerequisite you already have:** basic Java syntax, one-dimensional arrays, `ArrayList` basics, and file I/O

1. The short version

This unit is where the course turns a corner. Up to now, a program was mostly a sequence of steps operating on arrays and files. Starting this week, a program is also a collection of cooperating objects, each with its own data and its own behavior. We introduce the anatomy of a class (fields, a constructor, methods, getters and setters, and private helper methods), the three characteristics every object has (state, behavior, and identity), and how access modifiers control what other code is allowed to see or change.

Once you can build one class well, we spend the rest of the unit combining classes into small programs: creating several objects, passing them to methods, storing them in an `ArrayList`, and writing the three methods almost every class needs sooner or later, `toString`, `equals`, and `hashCode`. We close with Java interfaces, including default methods, and a short preview of inheritance, which is not part of this unit but gets its own unit later in the semester.

Class time is spent mostly in code. The slides give you vocabulary and motivation; the real learning happens while we build a running program together.

2. What We Will Cover

- **Why object-oriented programming.** Encapsulation, and why bundling data with the behavior that operates on it tends to produce code that is easier to change safely.
- **The three characteristics of an object.** State, behavior, and identity, and how each shows up in code.
- **Anatomy of a class.** Fields (data members), a constructor, methods, getters and setters, and private helper methods.
- **Access modifiers.** Public, private, and default (package-private), with a first look at protected.
- **Creating and using objects.** Instantiation, calling methods, and working with more than one object at a time.
- **Building a small program together.** Several classes cooperating, including how one object can depend on or contain another (dependence and aggregation).

- **Passing objects to methods.** What is actually copied when an object reference is passed as an argument.
- **Objects in an ArrayList.** Storing, searching, updating, and reporting on a list of objects rather than a list of primitives.
- **Overriding toString, equals, and hashCode.** Why the versions every class inherits by default are usually not the ones you want.
- **Interfaces.** Declaring one, implementing one, and writing a default method.
- **A preview of inheritance.** Just enough vocabulary (is-a versus has-a versus uses-a) to recognize it when the dedicated unit begins.

3. Objectives: By the End of These Five Days, You Should Be Able To

1. **Explain** the three characteristics of an object, state, behavior, and identity, in your own words.
2. **Write** a class with private fields, a constructor, getters and setters, and at least one private helper method.
3. **Instantiate** objects from a class and call their methods correctly, including working with several objects at once.
4. **Choose** the correct access modifier, public, private, or default, for a given field or method, and explain the choice.
5. **Build** a small program that creates and coordinates multiple classes, including at least one class that contains or depends on another.
6. **Pass** an object to a method and explain what is actually passed, and what is not.
7. **Store** objects in an **ArrayList** and iterate over the list to find, update, or report on the data it holds.
8. **Override** **toString**, **equals**, and **hashCode** for a class you write, and explain why the default versions usually fall short.
9. **Declare and implement** a Java interface, including a default method.
10. **Describe**, at a high level, what inheritance is and why the relationships you learn this unit (uses-a, has-a) set up the next one (is-a).

4. How to Show Up Ready

- Read *Core Java, Volume I*, Chapter 4 (Objects and Classes) and Chapter 6.1 (Interfaces), as listed on the course schedule, before the unit begins.
- Review your Weeks 1 through 3 material on arrays, **ArrayList**, and file I/O. This unit assumes all three are comfortable, since the programs we build reuse them.
- Bring a laptop, or plan to use a lab computer, since most of each class day happens live in code. Expect the pace to pick up quickly: we start with one small class and end the unit combining several classes, an **ArrayList**, and overridden methods into one program.